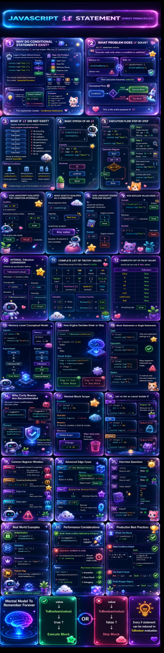
Hitesh Rathore (HR) Notes



JAVASCRIPT if STATEMENT (FIRST PRINCIPLES)

1 WHY DO CONDITIONAL STATEMENTS EXIST?

2 WHAT PROBLEM DOES if SOLVE?

"Execute code only when a condition is satisfied."

Without if:

```
withdrawMoney();
```

With if:

```
if (pinCorrect) {
  withdrawMoney();
}
```

Now execution becomes selective.

This is the entire purpose of if.

3 WHAT IF if DID NOT EXIST?

4 BASIC SYNTAX OF AN if

```
if (condition) {
  statements;
}
```

Example:

```
let age = 20;
if (age >= 18) {
  console.log("Adult");
}
```

5 EXECUTION FLOW STEP-BY-STEP

6 HOW JAVASCRIPT EVALUATES THE CONDITION INTERNALLY

7 WHAT EXACTLY QUALIFIES AS A CONDITION?

Any value

8 DOES JAVASCRIPT REQUIRE BOOLEAN VALUES?

9 NON-BOOLEAN VALUES INSIDE if

10 INTERNAL ToBoolean CONVERSION

11 COMPLETE LIST OF TRUTHY VALUES

12 COMPLETE LIST OF FALSY VALUES

Value	ToBoolean
false	false
0	false
-0	false
0n	false
""	false
null	false
undefined	false
NaN	false

13 Memory-Level Conceptual Model

14 How Engine Decides Enter or Skip

15 Block Statements vs Single Statements

16 Why Curly Braces Are Recommended

17 Nested Block Scope

18 var vs let vs const Inside if

19 Common Beginner Mistakes

20 Advanced Edge Cases

21 Interview Questions

22 Real World Examples

23 Performance Considerations

24 Production Best Practices

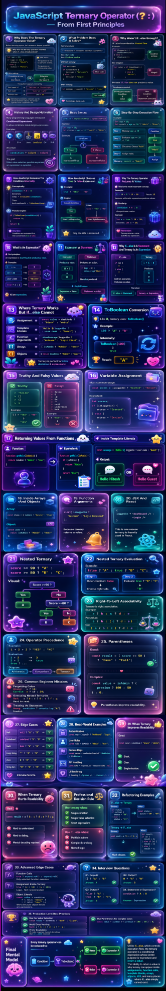
Mental Model To Remember Forever

value ↓ ToBoolean(value) ↓ true ? ↓ Execute Block

OR

value ↓ ToBoolean(value) ↓ false ? ↓ Skip Block

Every if statement can be reduced to ToBoolean evaluation.



JavaScript Ternary Operator (? :)
— From First Principles

1 Why Does The Ternary Operator Exist?

2 What Problem Does It Solve?

3 Why Wasn't if...else Enough?

4 History And Design Motivation

5 Basic Syntax

```
condition ? expressionIfTrue : expressionIfFalse
```

6 Step-By-Step Execution Flow

7 How JavaScript Evaluates The Condition Internally

8 How JavaScript Chooses True Or False Expression

9 Why The Ternary Operator Returns A Value

10 What Is An Expression?

11 Expression vs Statement

12 Why if...else Is A Statement And Ternary Is An Expression

13 Where Ternary Works But if...else Cannot

- Assignment
- Template Literals
- Function Arguments
- Arrays
- Objects

Ternary is perfect for inline decisions & expressions!

14 ToBoolean Conversion

Like if, ternary uses: ToBoolean()

Example:

```
100 ? "A" : "B"
```

Internally:

ToBoolean(100) ↓ true

Result: "A"

15 Truthy And Falsy Values

16 Variable Assignment

Most common usage.

```
const access = isLoggedIn ? "Granted" : "Denied";
```

Equivalent:

```
let access;
if (isLoggedIn) {
  access = "Granted";
} else {
  access = "Denied";
}
```

17 Returning Values From Functions

Inside Template Literals

Output:

Hello Hitesh OR Hello Guest

18. Inside Arrays And Objects

19. Function Arguments

20. JSX And React

This is one reason ternary is heavily used in React.

21 Nested Ternary

```
score >= 90 ? "A" :
score >= 80 ? "B" : "C";
```

22 Nested Ternary Evaluation

23 Right-To-Left Associativity

Ternary is right associative.

```
a ? b : c ? d : e
```

Parsed as:

```
a ? b : (c ? d : e)
```

24. Operator Precedence

25. Parentheses

Good:

```
const result = ( score >= 50 )
  ? "Pass" : "Fail";
```

Complex:

```
const value = isAdmin ? (
  premium ? 100 : 50
) : 0;
```

Parentheses improve readability.

26. Common Beginner Mistakes

27. Edge Cases

28. Real-World Examples

- Authentication
- User Roles
- Feature Flags
- API Handling
- UI Rendering

29. When Ternary Improves Readability

Good:

- Short.
- Clear.
- Single decision.

30. When Ternary Hurts Readability

- Hard to understand.
- Hard to debug.
- Mental decoding required.

31. Professional Decision Rule

Use ternary when:

- Single condition
- Single value selection
- Short expressions

Use if...else when:

- Multiple actions
- Complex branching
- Nested logic

32. Refactoring Examples

33. Advanced Edge Cases

34. Interview Questions

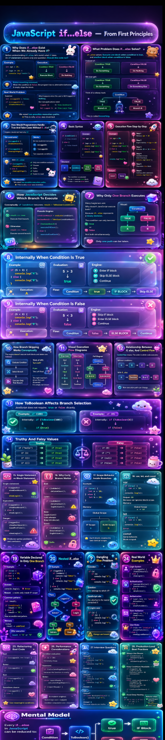
35. Production-Level Best Practices

Final Mental Model

Every ternary operator can be reduced to:

Condition → ToBoolean() → true → Expression A / false → Expression B

Unlike if...else, which controls execution flow, the ternary operator is a conditional expression whose entire purpose is to produce and return a value. That ability to return a value is why ternary can appear inside assignments, function calls, template literals, arrays, objects, JSX, and many places where if...else simply cannot exist.



JavaScript if...else — From First Principles

1 Why Does if...else Exist When We Already Have if?

2 What Problem Does if...else Solve?

3 How Would Programmers Handle True And False Cases Without if...else?

4 Basic Syntax

5 Execution Flow Step-by-Step

6 How JavaScript Decides Which Branch To Execute

7 Why Only One Branch Executes

8 Internally When Condition Is True

Example:

```
if (5 > 3) {
  console.log("A");
} else {
  console.log("B");
}
```

Evaluation: 5 > 3 ↓ true

Engine:

- Enter IF block
- Skip ELSE block
- Continue

Execution: A

Flow: Condition → true → IF BLOCK → Skip ELSE

9 Internally When Condition Is False

Example:

```
if (5 < 3) {
  console.log("A");
} else {
  console.log("B");
}
```

Evaluation: 5 < 3 ↓ false

Engine:

- Skip IF block
- Enter ELSE block
- Continue

Execution: B

Flow: Condition → false → ELSE BLOCK → Continue

10 How Branch Skipping Works Conceptually

11 Visual Execution Flow Diagrams

12 Relationship Between if, else, And Control Flow

13 How ToBoolean Affects Branch Selection

14 Truthy And Falsy Values

15 Single Statement vs Block Statements

16 Why Curly Braces Matter

17 Scope Behavior Inside Branches

18 var, let, and const

19 Variable Declared In Only One Branch

20 Nested if...else

21 Dangling Else Problem

24 Real World Examples

25 Refactoring Bad Code

26 Performance Considerations

27 Interview Questions

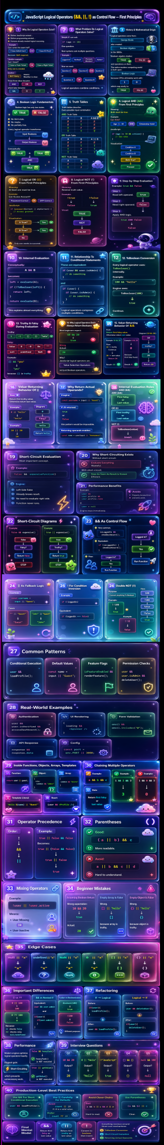
28 Production-Level Best Practices

Mental Model

Every if...else in JavaScript can be reduced to:

Condition → ToBoolean() → true → IF Block / false → ELSE Block

---



JavaScript Logical Operators (&&, ||, !) as Control Flow — First Principles

1 Why Do Logical Operators Exist?

2 What Problem Do Logical Operators Solve?

3 History & Mathematical Origin

4 Boolean Logic Fundamentals

5 Truth Tables

6 Logical AND (&&) From First Principles

7 Logical OR (||) From First Principles

8 Logical NOT (!) From First Principles

9 Step-by-Step Evaluation

10 Internal Evaluation

11 Relationship To Conditional Statements

12 ToBoolean Conversion

13 Truthy & Falsy During Evaluation

14 Why Logical Operators Don't Always Return Booleans

15 Value-Returning Behavior Of &&

16 Value-Returning Behavior Of ||

17 Why Return Actual Operands?

18 Internal Evaluation Rules

19 Short-Circuit Evaluation

20 Why Short-Circuiting Exists

21 Performance Benefits

22 Short-Circuit Diagrams

23 && As Control Flow

24 || As Fallback Logic

25 ! For Condition Inversion

26 Double NOT (!!)

27 Common Patterns

Conditional Execution

```
user &&
loadProfile();
```

Default Values

```
const name =
input || "Guest";
```

Feature Flags

```
isFeatureEnabled &&
renderFeature();
```

Permission Checks

```
user &&
user.isAdmin &&
deleteUser();
```

28 Real-World Examples

Authentication

UI Rendering

```
loading &&
  <Spinner />
```

Form Validation

```
email &&
email.includes("@");
```

API Response

```
response &&
response.data
```

Config

```
const port =
  env.PORT || 3000;
```

29 Inside Functions, Objects, Arrays, Templates

30 Chaining Multiple Operators

Example: a && b && c

Example: 1 && 2 && 3 ↓ 3

Example: 1 && 0 && 3 ↓ 0

Rule: Return first falsy or last value

31 Operator Precedence

Example:

```
true || false && false
```

Becomes:

```
true || (false && false)
↓
true || false
↓
true
```

32 Parentheses

Good:

```
( a || b ) && c
```

More readable.

Avoid:

```
a || b && c || d
```

Hard to understand.

33 Mixing Operators

34 Beginner Mistakes

Assuming Boolean Return

Wrong expectation: 10 && 20 ↓ true

Actual: 20

Empty Array Is False

Wrong. [] || "hello" ↓ [] because array is truthy.

Empty Object Is False

Wrong. {} || "hello" ↓ {} because object is truthy.

35 Edge Cases

- null || "x" ↓ "x"
- undefined || "x" ↓ "x"
- NaN || "x" ↓ "x"
- 0 || "x" ↓ "x"
- [] || "x" ↓ []
- {} || "x" ↓ {}

36 Important Differences

37 Refactoring

if → Logical

Logical → if

38 Performance

39 Interview Questions

Q1	Q2	Q3	Q4	Q5
10 && 20	0 || "Hello"	!!"JavaScript"	[] && {}	null && 100
Output?	Output?	Output?	Output?	Output?
20	"Hello"	true	{}	null

40 Production-Level Best Practices

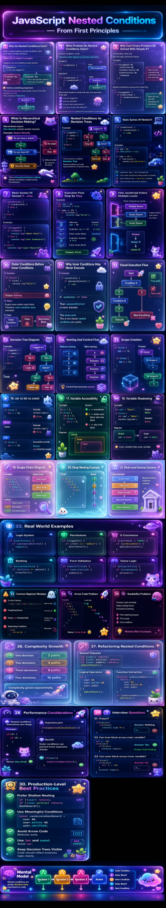
- Use && For Short Conditional Execution
- Use || Carefully For Defaults
- Avoid Clever Chains
- Use Parentheses

Final Mental Model

- &&: Return first falsy or last value
- ||: Return first truthy or last value
- !: Convert to boolean then invert

Everything revolves around these core mechanisms:

1. ToBoolean Conversion
2. Short-Circuit Evaluation
3. Returning Actual Operands



JavaScript Nested Conditions
— From First Principles

1 Why Do Nested Conditions Exist?

2 What Problem Do Nested Conditions Solve?

3 Why Can't Every Problem Be Solved With Simple if?

4 What Is Hierarchical Decision Making?

5 Nested Conditions As Decision Trees

6 Basic Syntax Of Nested if

7 Basic Syntax Of Nested if...else

8 Execution Flow Step By Step

9 How JavaScript Enters Multiple Levels

10 Outer Conditions Before Inner Conditions

11 Why Inner Conditions May Never Execute

12 Visual Execution Flow

13 Decision Tree Diagram

14 Nesting And Control Flow

15 Scope Creation

16. var vs let vs const

17. Variable Accessibility

18. Variable Shadowing

19. Scope Chain Diagram

20. Deep Nesting Example

21. Multi-Level Decision Systems

22. Real World Examples

- Login System
- Permissions
- E-Commerce
- Banking
- Form Validation
- Game Logic

23 Common Beginner Mistakes

24 Arrow Code Problem

25 Readability Problems

26. Complexity Growth

One decision:	2 paths
Two decisions:	4 paths
Three decisions:	8 paths
Four decisions:	16 paths

Complexity grows exponentially.

27. Refactoring Nested Conditions

- Guard Clauses
- Logical Operators
- Function Extraction

28 Performance Considerations

29 Interview Questions

30. Production-Level Best Practices

- Prefer Shallow Nesting
- Use Meaningful Conditions
- Avoid Arrow Code: Refactor early.
- Use let and const: Avoid var.
- Keep Decision Trees Visible: Code should reflect business logic clearly.

Mental Model

Nested conditions are simply decision trees expressed as code.

Decision 1 → Decision 2 → Decision 3 → Action

1. Outer Condition
2. Enter Block?
3. Inner Condition
4. Enter Block?
5. Next Condition